西北农林科技大学

Northwest A&F University

流式计算实验报告

姓　名：	陈
学　号：	202101
班　级：	数科 2102
时　间：	2024 年 4 月 18 日

实验 2:Scala 基础——变量定义与基本类型

一、实验目的

学习 scala 的变量定义与基本类型

二、实验步骤

1、定义变量

```
scala> val x = 1
val x: Int = 1

scala> var y = 2
var y: Int = 2

scala> val msg = "Hello, world!"
val msg: String = Hello, world!

scala> var x = 1
var x: Int = 1

scala> x = 10
// mutated x

scala> x = "abc"
           ^
       error: type mismatch;
        found   : String("abc")
        required: Int

scala>

scala> val x = 1
val x: Int = 1

scala> x = 10
         ^
       error: reassignment to val

scala> val a, b, c = 1
val a: Int = 1
val b: Int = 1
val c: Int = 1

scala> val x = 1
val x: Int = 1

scala> val x = 10
val x: Int = 10

scala> x
val res0: Int = 10
```

2、Scala 的基本类型

2.1 整数字面量

```
scala> val a = 100
val a: Int = 100

scala> val b = 0x123Abc
val b: Int = 1194684

scala> val c: Byte = 100
val c: Byte = 100

scala> val d = 200L
val d: Long = 200
```

2.2 浮点数字面量

```
scala> val a = 1.2e3
val a: Double = 1200.0

scala> val b = -3.2f
val b: Float = -3.2

scala> val c: Float = -3.2f
val c: Float = -3.2

scala> val d: Double = -3.2F
val d: Double = -3.200000047683716
```

2.3 字符与字符串字面量

```
scala> val a = 'A'
val a: Char = A

scala> val b = '\u0041'
val b: Char = A

scala> val c = '\u0042'
val c: Char = B

scala> val AB = 1
val AB: Int = 1

scala> val d = '\\'
val d: Char = \
```

```
scala> val a = "\\\\\\"
val a: String = \\\

scala> val b = """So long \u0041 String \\\'\"!"""
                          ^
       warning: Unicode escapes in triple quoted str
val b: String = So long A String \\\'\"!
```

2.4 字符串插值

```
scala> val name = "ABC"
val name: String = ABC

scala> println(s"$name DEFG")
ABC DEFG

scala> s"Sum = ${1+10}"
val res1: String = Sum = 11

scala> s"\\\\"
val res2: String = \\

scala> raw"\\\\"
val res3: String = \\\\

scala> printf(f"${math.Pi}%.5f")
3.14159
```

三、实验任务

```
scala> var a = 1
var a: Int = 1

scala> val b = 2
val b: Int = 2

scala> var c = 3
var c: Int = 3

scala> var d = 4
var d: Int = 4

scala> var e = 5
var e: Int = 5

scala> e = 15
// mutated e
```

```
scala> val a = 1e10
val a: Double = 1.0E10

scala> val b = 1e-2
val b: Double = 0.01

scala> var c = 1.14514f
var c: Float = 1.14514

scala> var d = 1.141514F
var d: Float = 1.141514

scala> var d: Double = 1.14514
var d: Double = 1.14514

scala> var d: Double = 1.14514F'
                               ^
       error: unclosed character literal

scala> var d: Double = 1.14514F
var d: Double = 1.1451400518417358
```

```
scala> val a = 'A'
val a: Char = A

scala> val b = '\u0080'
val b: Char = ▯

scala> val b = '\u0050'
val b: Char = P

scala> val d = "\'\\"
val d: String = '\

scala> val str = "\"Genshin Start\"!"
val str: String = "Genshin Start"!
```

```
scala> val a = "Genshin"
val a: String = Genshin

scala> val b = "Start"
val b: String = Start

scala> println(s"$a $b")
Genshin Start

scala> s"Product = ${9*8}"
val res14: String = Product = 72

scala> raw"""\\"\"\"\"""
val res15: String = \\"\"\"\

scala> printf(f"${math.acos(-1)%.6f")
                                    ^
       error: unclosed string literal

scala> printf(f"${math.acos(-1)}%.6f")
3.141593
```

实验 3：函数及其几种形式

一、实验目的

学习 Scala 中函数及其几种形式，并完成指定任务

二、 实验步骤

1、max 函数的实现

```
USER_ID: 0, GROUP_ID: 0
root@cg:~# cd /headless/Desktop/Scala/ex1
root@cg:~/Desktop/Scala/ex1# ls
Hello.class  hello.scala
root@cg:~/Desktop/Scala/ex1# scala hello.scala
Hello, world!
root@cg:~/Desktop/Scala/ex1# cd ..
root@cg:~/Desktop/Scala# cd ex3
root@cg:~/Desktop/Scala/ex3# scala
Welcome to Scala 2.13.3 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_271).
Type in expressions for evaluation. Or try :help.

scala> def max(x: Int, y: Int): Int = {
     | if (x > y)
     |     x
     | else
     |     y
     | }
def max(x: Int, y: Int): Int

scala> max(2, 3)
val res0: Int = 3
```

2、函数返回结果

```
scala> def max(x: Int, y: Int): Int = {
     | if (x > y)
     |     x
     | else
     |     y
     | }
def max(x: Int, y: Int): Int

scala> max(2, 3)
val res0: Int = 3

scala>

scala> def add(x:Int, y:Int) = {x + y}
def add(x: Int, y: Int): Int

scala> add(2, 3)
val res1: Int = 5

scala> def nothing(x:Int, y:Int): Unit = {x + y}
def nothing(x: Int, y: Int): Unit

scala> nothing(1, 2)

scala>
```

3、嵌套函数

```
scala> nothing(1, 2)

scala> def addSub(x: Int, y: Int) = {
     |     def sub(z: Int) = z - 10
     |     if (x > y) sub(x-y) else sub(y-x)
     | }
def addSub(x: Int, y: Int): Int

scala> addSub(100, 20)
val res3: Int = 70

scala> addsub(20, 100)
       ^
       error: not found: value addsub

scala> def addSub(x: Int, y: Int) = {
     |     def sub(z: Int) = z - 10
     |     if (x > y) sub(x-y) else sub(y-x)
     | }
def addSub(x: Int, y: Int): Int

scala> addSub(100, 20)
val res5: Int = 70

scala> addSub(20, 100)
val res6: Int = 70

scala>
```

4、函数字面量

```
scala> val f = ( _: Int) + ( _:Int)
val f: (Int, Int) => Int = $Lambda$1002/2090144454@4f65af91

scala> f(1, 2)
val res7: Int = 3

scala> val add = (x: Int) => { (y: Int) => x + y }
val add: Int => (Int => Int) = $Lambda$1018/2014583896@9863ed5

scala> add(2)(10)
val res8: Int = 12

scala> def aFunc(f: Int => Int) = f(1) + 1
def aFunc(f: Int => Int): Int

scala> aFunc(x => x + 1)
val res9: Int = 3

scala>
```

5、部分应用函数

```
scala> def sum(x: Int, y: Int, z: Int) = x + y + z
def sum(x: Int, y: Int, z: Int): Int

scala> val a = sum(1, 2, 3)
val a: Int = 6

scala> val b = sum(1, _: Int, 3)
val b: Int => Int = $Lambda$1030/487053096@6d98e607

scala> b(2)
val res10: Int = 6

scala> val c = sum _
val c: (Int, Int, Int) => Int = $Lambda$1032/491879@214738c1

scala> c(1, 2, 3)
val res11: Int = 6

scala>
```

```
scala> def needSum(f: (Int, Int, Int) => Int) = f(1, 2, 3)
def needSum(f: (Int, Int, Int) => Int): Int

scala> needSum(sum _)
val res12: Int = 6

scala> needSum(sum)
val res13: Int = 6
```

6、闭包

```
scala> var more = 1
var more: Int = 1

scala> val addMore = (x: Int) => x + more
val addMore: Int => Int = $Lambda$1038/1919759582@162796e1

scala> addMore(1)
val res18: Int = 2

scala> more = 2
// mutated more

scala> addMore(1)
val res19: Int = 3

scala> var more = 10
var more: Int = 10

scala> addMore(1)
val res20: Int = 3

scala> more = -114514
// mutated more

scala> addMore(1)
val res21: Int = 3

scala>
```

7、函数特殊调用形式

7.1 具名参数

```
scala> def max(x: Int, y: Int, z: Int) = {
     |     if (x > y && x > z) println("x is maximum")
     |     else if (y > x && y > z) println("y is maximum")
     |     else println("z is maximum)
     |     else println("z is maximum)
                      ^
On line 4: error: unclosed string literal

scala> }
       ^
       error: eof expected but '}' found.

scala> def max(x: Int, y: Int, z: Int) = {
     |     if (x > y && x > z) println("x is maximum")
     |     else if (y > x && y > z) println("y is maximum")
     |     else println("z is maximum")
     | }
def max(x: Int, y: Int, z: Int): Unit

scala> max(1, z = 10, y = 100)
y is maximum
```

7.2 默认参数值

```
scala> def max(x: Int, y: Int, z: Int) = {
     |     if (x > y && x > z) println("x is maximum")
     |     else if (y > x && y > z) println("y is maximum")
     |     else println("z is maximum")
     | }
def max(x: Int, y: Int, z: Int): Unit

scala> max(1, z = 10, y = 100)
y is maximum

scala> max(1,1,1)
z is maximum

scala> def max(x: Int = 10, y: Int, z: Int) = {
     |     if (x > y && x > z) println("x is maximum")
     |     else if (y > x && y > z) println("y is maximum")
     |     else println("z is maximum")
     | }
def max(x: Int, y: Int, z: Int): Unit

scala> max(y = 3, z = 5)
x is maximum
```

7.3 重复参数

```
scala> def addMany(msg: String, num: Int*) = {
     |     var sum = 0
     |     for(x <- num) sum += x
     |     println(msg + sum)
     | }
def addMany(msg: String, num: Int*): Unit

scala> addMany("sum = ", 1, 2, 3)
sum = 6

scala> addMany("sum = ")
sum = 0

scala> addMany("sum = ", Array(1, 2, 3): _*)
                         ^
       warning: Passing an explicit array value to a Scala varargs method is deprecated (since
 2.13.0) and will result in a defensive copy; Use the more efficient non-copying ArraySeq.unsa
feWrapArray or an explicit toIndexedSeq call
sum = 6
```

8、柯里化

```
scala> def add(x: Int, y: Int, z: Int) = x + y + z
def add(x: Int, y: Int, z: Int): Int

scala> add(1, 2, 3)
val res28: Int = 6

scala> def addCurry(x: Int)(y: Int)(z: Int) = x + y + z
def addCurry(x: Int)(y: Int)(z: Int): Int

scala> addCurry(1)(2) {3}
val res29: Int = 6
```

9、传名参数

```
scala> def myAssert(predicate: () => Boolean) =
     |     if (assertionEnabled && !predicate())
     |         throw new AssertionError
def myAssert(predicate: () => Boolean): Unit

scala> myAssert(() => 5 > 3)

scala> def byNameAssert(predicate: => Boolean) =
     |     if (assertionEnabled && !predicate)
     |         throw new AssertionError
def byNameAssert(predicate: => Boolean): Unit

scala> byNameAssert(5 > 3)

scala> def boolAssert(predicate: Boolean) =
     |     if (assertionEnabled && !predicate)
     |         throw new AssertionError
def boolAssert(predicate: Boolean): Unit

scala> boolAssert(5 > 3)
```

```
scala> myAssert(() => 5/0 == 0)

scala> byNameAssert(5/0 == 0)

scala> boolAssert(5/0 == 0)
java.lang.ArithmeticException: / by zero
  ... 32 elided

scala> assertionEnabled = true
// mutated assertionEnabled

scala> myAssert(() => 5/0 == 0)
java.lang.ArithmeticException: / by zero
  at $anonfun$res42$1(<console>:1)
  at myAssert(<console>:2)
  ... 32 elided

scala> byNameAssert(5/0 == 0)
java.lang.ArithmeticException: / by zero
  at $anonfun$res43$1(<console>:1)
  at byNameAssert(<console>:2)
  ... 32 elided

scala> boolAssert(5/0 == 0)
java.lang.ArithmeticException: / by zero
  ... 32 elided
```

三、实验任务

```
scala> def judge(n: Int): Boolean = {
     |     def powOfThree(num: Int): Int = num*num*num
     |     var cnt = 0
     |     var tmp = n
     |     var sum = n
     |     while(tmp != 0) {
     |         sum -= powOfThree(tmp%10)
     |         tmp = tmp/10
     |         cnt = cnt + 1
     |     }
     |     if (cnt != 3 || sum != 0)
     |         false
     |     else
     |         true
     | }
def judge(n: Int): Boolean

scala> judge(10)
val res45: Boolean = false

scala> judge(153)
val res46: Boolean = true

scala> judge(370)
val res47: Boolean = true

scala> judge(371)
val res48: Boolean = true

scala> judge(407)
val res49: Boolean = true
```

实验 4：类和对象

一、实验目的

掌握 scala 定义类的方法，掌握 scala 对象

二、实验步骤

1、类

```
scala> class Students{
     |     var name = "None"
     |     def register(n: String) = name = n
     | }
class Students

scala> val stu = new Students
val stu: Students = Students@10f19647

scala> stu.name
val res0: String = None

scala> stu.register("Skichio")

scala> stu.name
val res2: String = Skichio

scala> stu = new Students
           ^
       error: reassignment to val
```

```
scala> class Students {
     |     private var name = "None"
     |     def register(n: String) = name = n
     |     def getName() = println(name)
     | }
class Students

scala> val stu = new Students
val stu: Students = Students@59d77850

scala> stu.register("Skichio")

scala> stu.name
           ^
       error: variable name in class Students c

scala> stu.getName
           ^
       warning: Auto-application to `()` is dep
 method getName,
       or remove the empty argument list from :
       In Scala 3, an unapplied method like thi
Skichio
```

2、类的构造方法

2.1 主构造方法

```
scala> class Students(n:String){
     |     val name = n
     |     println("A student named " + n + " has been registered.")
     | }
class Students

scala> val stu = new Students("Skichio")
A student named Skichio has been registered.
val stu: Students = Students@4ae49387
```

2.2 辅助构造方法

```
scala> class Students(n:String){
     |     val name = n
     |     def this() = this("None")
     |     println("A student named " + n + " has been registered.")
     | }
class Students

scala> val stu = new Students
A student named None has been registered.
val stu: Students = Students@66abb2fa
```

2.3 私有主构造方法

```
scala> class Students private (n: String, m: Int){
     |     val name = n
     |     val score = m
     |     def this(n:String) = this(n,100)
     |     println(n + "'s score is " + m)
     | }
class Students

scala> val stu = new Students("Bill", 90)
                                      ^
       error: too many arguments (found 2, expected 1)

scala> val stu = new Students("Bill")
Bill's score is 100
val stu: Students = Students@1734b1a
```

3、重写 toString 方法

```
scala> class Students(n: String){
     |     val name = n
     |     override def toString = "A student named " + n + "."
     | }
class Students

scala> val stu = new Students("Skichio")
val stu: Students = A student named Skichio.
```

4、类参数

```
scala> class Students(val name:String, var score:Int){
     |     def exam(s: Int) = score = s
     |     override def toString = name + "'s score is " + score + "."
     | }
class Students

scala> val stu = new Students("Tim", 90)
val stu: Students = Tim's score is 90.

scala> stu.exam(100)

scala> stu.score
val res4: Int = 100
```

5、单例对象与伴生对象

```
scala> class A {val a = 10}
class A

scala> val x = new A
val x: A = A@3d5790ea

scala> x.a
val res0: Int = 10

scala> (new A).a
val res1: Int = 10

scala> object B {val b = "a singleton object"}
object B

scala> B.b
val res2: String = a singleton object

scala> val y = B
val y: B.type = B$@6121a7dd

scala> y.B
         ^
       error: value B is not a member of object

scala> y.b
val res4: String = a singleton object
```

```
scala> object X
object X

scala> object Y
object Y

scala> var x = X
var x: X.type = X$@2ab7f649

scala> x = Y
           ^
       error: type mismatch;
        found   : Y.type
        required: X.type
```

6、工厂对象与工厂方法

```
root@cg:~/Desktop/Scala/ex4# scalac students.scala
root@cg:~/Desktop/Scala/ex4# scala
Welcome to Scala 2.13.3 (Java HotSpot(TM) 64-Bit Server VM, J
Type in expressions for evaluation. Or try :help.

scala> import Students._
import Students._

scala> val stu = registerStu("Skichio", 100)
val stu: Students = Skichio's score is 100.
```

7、apply 方法

```
root@cg:~/Desktop/Scala/ex4# scalac students2.scala
root@cg:~/Desktop/Scala/ex4# scala
Welcome to Scala 2.13.3 (Java HotSpot(TM) 64-Bit Ser
Type in expressions for evaluation. Or try :help.

scala> val stu2 = Students2("Skichio", 60)
val stu2: Students2 = Skichio's score is 60.

scala> stu2(80)

scala> stu2.display()
Current score is 80.
```

8、主函数

```
root@cg:~/Desktop/Scala/ex4# scalac main.scala
root@cg:~/Desktop/Scala/ex4# scala Start Tom
Arguments are deficient!
root@cg:~/Desktop/Scala/ex4# scala Start Tom aa
Second argument must be a Int!
root@cg:~/Desktop/Scala/ex4# scala Start Tom 100
Tom's score is 100.
```

```
object Start2 extends App {
    try {
        var sum = 0
        for (arg <- args) {
            sum += arg.toInt
        }
        println("sum = " + sum)
    } catch {
        case ex: NumberFormatException => println("Arguments must be Int!")
    }
}
```

```
root@cg:~/Desktop/Scala/ex4# vim main2.scala
root@cg:~/Desktop/Scala/ex4# scalac main2.scala
root@cg:~/Desktop/Scala/ex4# scala Start2 10 -8 20 AAA
Arguments must be Int!
root@cg:~/Desktop/Scala/ex4# scala Start2 10 -8 20 8
sum = 30
root@cg:~/Desktop/Scala/ex4#
```

三、实验任务

1、老师类

```
class Students(val name: String, var score: Int) {
    def display() = println(name + "'s score is " + score)
    def apply(s: Int):Unit = score = s
}

object Students {
    def apply(name: String, score: Int) = new Students(name, score)
}

class Teacher(val name: String) {
    def updateScore(obj: Students, score: Int): Students = {
        obj(score)
        obj
    }
}

object Teacher {
    def apply(name: String) = new Teacher(name)
}

object Start {
    def main(args: Array[String]) = {
        try {
            var stu = Students("Skichio", 90)
            stu.display()
            val teacher = Teacher("Mike")
            val score = args(0).toInt
            stu = teacher.updateScore(stu, score)
            stu.display()
        } catch {
            case ex: ArrayIndexOutOfBoundsException => println("Arguments are deficient!")
            case ex: NumberFormatException => println("First argument must be a Int!")
        }
    }
}
```

```
root@cg:~/Desktop/Scala/ex4# scalac work1.scala
root@cg:~/Desktop/Scala/ex4# scala Start 100
Skichio's score is 90
Skichio's score is 100
```

2、圆类

```
import io.StdIn._

class Point(var x: Int, var y: Int) {
    def getDis(p: Point) = {
        math.sqrt(math.pow(x-p.x,2)+math.pow(y-p.y,2))
    }
}

object Point {
    def apply(x: Int, y: Int) = new Point(x, y)
}

class Circle(val o: Point, val r: Int) {
    def judge(c2: Circle) = {
        if(o.getDis(c2.o) < r+c2.r) 1
        else 0
    }
}

object Circle {
    def apply(x: Int, y: Int, r: Int): Circle = {
        val p = Point(x, y)
        new Circle(p, r)
    }
}

object Start {
    def main(args: Array[String]) = {
        println("Enter the coordinates and radius of the center of the two circles")
        var x1 = readInt()
        var y1 = readInt()
        var r1 = readInt()
        var x2 = readInt()
        var y2 = readInt()
        var r2 = readInt()

        val c1 = Circle(x1,y1,r1)
        val c2 = Circle(x2,y2,r2)
        if(c1.judge(c2) == 1) println("These two circles intersert")
        else println("These two circles don't intersert")
    }
}
```

圆心在（1，1）与圆心在（4，5）的两个圆，半径分别为 1，4 时刚好相切，不相交；半径为 1，5 时这两个圆相交

```
root@cg:~/Desktop/Scala/ex4# scala Start
Enter the coordinates and radius of the center of the two circles
1
1
1
4
5
4
These two circles don't intersert
root@cg:~/Desktop/Scala/ex4# scala Start
Enter the coordinates and radius of the center of the two circles
1
1
1
4
5
5
```

实验 5：操作符即方法

一、实验目的

掌握 scala 的常用操作符、掌握 scala 对象相等性

二、实验步骤

1、三种操作符

1.1 前缀操作符

```
scala> class MyInt(val x: Int) {
     |     def unary_! = -x
     |     def unary_* = x*2
     | }
class MyInt

scala> val mi = new MyInt(10)
val mi: MyInt = MyInt@2d82408

scala> !mi
val res0: Int = -10

scala> *mi
       ^
       error: not found: value *
        ^
       error: postfix operator mi needs to be enabled
       by making the implicit value scala.language.postfixOps visible.
       This can be achieved by adding the import clause 'import scala.language.postfixOps'
       or by setting the compiler option -language:postfixOps.
       See the Scaladoc for value scala.language.postfixOps for a discussion
       why the feature needs to be explicitly enabled.

scala> \*mi
       ^
       error: not found: value \*
         ^
       error: postfix operator mi needs to be enabled
       by making the implicit value scala.language.postfixOps visible.
       This can be achieved by adding the import clause 'import scala.language.postfixOps'
       or by setting the compiler option -language:postfixOps.
       See the Scaladoc for value scala.language.postfixOps for a discussion
       why the feature needs to be explicitly enabled.

scala> mi.unary_*
val res3: Int = 20
```

1.2 中缀操作符

```
scala> class MyInt2(val x: Int) {
     |     def +*(y: Int) = (x+y)*y
     |     def +:(y: Int) = x+y
     | }
class MyInt2

scala> val mi2 = new MyInt2(10)
val mi2: MyInt2 = MyInt2@19fbc594

scala> mi2 +* 10
val res4: Int = 200

scala> 10 +: mi2
val res5: Int = 20

scala> Console println "Genshin Start!"
Genshin Start!
```

1.3 后缀操作符

```
scala> class MyInt3(val x: Int) {
     |     def display() = println("The value is " + x + ".")
     | }
class MyInt3

scala> val mi3 = new MyInt3(10)
val mi3: MyInt3 = MyInt3@3e5c7588

scala> import scala.language.postfixOps
import scala.language.postfixOps

scala> mi3 display
           ^
       warning: Auto-application to `()` is deprecated. Supply the empty argu
d display,
       or remove the empty argument list from its definition (Java-defined me
       In Scala 3, an unapplied method like this will be eta-expanded into a
The value is 10.
```

2、对象的相等性

```
scala> val a = List(1, 0, -1)
val a: List[Int] = List(1, 0, -1)

scala> val b = List(1, 0, -1)
val b: List[Int] = List(1, 0, -1)

scala> val c = List(1, 0, 1)
val c: List[Int] = List(1, 0, 1)

scala> val d = a
val d: List[Int] = List(1, 0, -1)

scala> a == c
val res8: Boolean = false

scala> a == b
val res9: Boolean = true

scala> a equals b
val res10: Boolean = true

scala> a eq b
val res11: Boolean = false

scala> a eq d
val res12: Boolean = true
```

三、实验任务

```
scala> class MyInt(val x: Int) {
     |     def +(y: Int) = x+y
     |     def +(y: MyInt) = x+y.x
     |     def -(y: Int) = x-y
     |     def -(y: MyInt) = x-y.x
     |     def /(y: Int) = x/y
     |     def /(y: MyInt) = x/y.x
     |     def *(y: Int) = x*y
     |     def *(y: MyInt) = x*y.x
     |     def %(y: Int) = x%y
     |     def %(y: MyInt) = x%y.x
     | }
class MyInt

scala> val a = new MyInt(4)
val a: MyInt = MyInt@193594ad

scala> val b = new MyInt(3)
val b: MyInt = MyInt@fce6355

scala> val c = new MyInt(2)
val c: MyInt = MyInt@63d93e91

scala> a / b
val res13: Int = 1

scala> a / c
val res14: Int = 2

scala> a % c
val res15: Int = 0

scala> a % 5
val res16: Int = 4

scala> a * c
val res17: Int = 8

scala> a - c
val res18: Int = 2

scala> a + c
val res19: Int = 6

scala> a +3
val res20: Int = 7
```